

PolluSenseCheck: Cost Effective TinyML-based Air Quality Monitoring System with In-Built Sensor Fault Detection

Arko Datta*, Joyita Chakraborty*, Pabitra Majhi*, Astom Mondal[†], Satyaki Roy[‡], Subrata Nandi*, Sujoy Saha*

*Department of Computer Science and Engineering, National Institute of Technology, Durgapur, India
 {contact.arkodatta, joyita.ckra, officialpabitrarnajhi, subrata.nandi, sujoy.ju}@gmail.com

[†]Department of Computer Science and Engineering, Dr. B. C. Roy Engineering College, Durgapur, India
 astommondal013@gmail.com

[‡]Department of Mathematical Sciences, University of Alabama in Huntsville, Alabama, USA
 sr0215@uah.edu

Abstract—Air pollution is a significant contributor to respiratory diseases, posing severe health risks globally. Traditional air quality monitoring stations, while effective, are too expensive, particularly in developing nations like India, where numerous monitoring sites may be required to cover diverse urban and industrial regions. Existing cost-effective air quality monitoring systems are rendered inadequate by their reliance on sensors that are prone to drift, which necessitates frequent recalibration. This study addresses the gap by developing a cost-effective air quality monitoring device capable of detecting particulate matter, carbon monoxide (CO), temperature, and humidity, and incorporating self-diagnostic capabilities. It enhances reliability through a dual-sensing mechanism: it calculates the Air Quality Index (AQI) directly and uses a TinyML model to provide secondary-sensed AQI estimations. Deviations between these two AQI values signal sensor malfunction, which is tracked by a proposed algorithm, *Intercept*, that assigns fault scores to the sensors. Models were created for eight cities, showing accuracies from 0.69 – 0.89 for dust and 0.76 – 0.95 for CO. These models were optimized for real-time performance by converting them into TensorFlow Lite micro-models, allowing deployment on the device itself for continuous, autonomous sensor health diagnostics and improved long-term air quality management systems.

Index Terms—Air Quality, Sensor Drift, TinyML, Cost Effective Air Quality Monitoring Device

I. INTRODUCTION

Air pollution poses a significant health risk, particularly in urban areas where populations are exposed to air that frequently exceeds European and WHO air quality standards. Over the last decade, research has shown that particulate matter (PM) air pollution is linked to various health issues, with data suggesting that harmful effects can occur even at low exposure levels [1]. Studies assessing the effectiveness of air quality interventions, such as measures to reduce traffic congestion or industrial emissions, have documented improvements in air quality and corresponding health benefits, including reduced cardiovascular and respiratory disease [2]. A review of intervention studies has reported that efforts to lower air pollution levels have been associated with a decline in mortality and morbidity, particularly from cardiovascular and

respiratory conditions [3]. These studies lend insight into the complex relationship between air quality and health outcomes.

Integrating sensing technologies and machine learning approaches has shown great promise in monitoring air quality and mitigating its harmful effects. Such systems enable real-time data collection, which is essential given the variability of air pollution levels due to factors like traffic density and weather patterns [4]. Countries like India, where air pollution poses a significant public health risk benefit from such innovations. By adopting necessary improvements in monitoring infrastructure that combine pollutant concentration data with healthcare tools and predictive analytics, it is possible to reduce the burden of pollution-related diseases. These machine learning-driven systems improve upon traditional monitoring by analyzing the data more effectively, preempting bad air quality, and preventing adverse health outcomes [5], [6].

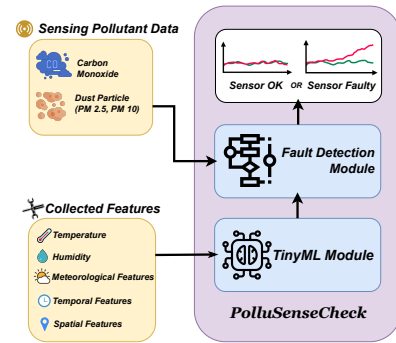


Fig. 1: Concept diagram of PolluSenseCheck

Environmental sensing increasingly relies on low-cost sensors paired with machine learning to monitor air quality and predict pollution (see section II). While affordable and easy to deploy, these systems often need frequent calibration and struggle with accuracy, especially in polluted urban or industrial areas. Timely and accurate air quality data is crucial for public health, policy decisions, and individual actions. To address inaccuracies, a promising solution involves using

established environmental markers to create detailed pollutant profiles. Integrating these profiles with sensor advancements can enhance low-cost systems for more reliable air quality management [7]–[9], enabling proactive pollution control and reducing health risks in both urban and rural areas. Currently, cost-effective air quality monitoring systems require periodic calibration against high-cost sensors to check their health. This process involves extra hardware and is often too technical for non-expert users, rendering these devices unreliable over time. This research proposes a TinyML-based embedded sensor health self-check algorithm to address this issue. By deploying intelligent algorithms on the edge, the device remains reliable in challenging networking environments, reduces network bandwidth usage, and eliminates server costs.

Key contributions. We make the following contributions.

1. Secondary sensing model. We develop two machine learning models, one for each pollutant, namely, dust particles and Carbon Monoxide (CO), two major contributors of outdoor air pollution. These models classify the Air Quality Index (AQI) on a scale of 1-6 for dust and 1-3 for CO. They utilize 53 features, engineered from local temperature, humidity, time, and GPS location, including meteorological factors (e.g., wind speed, barometric pressure, rain), temporal features (month, weekday), and land cover from landmarks. Subsequently, these features are scaled and transformed into 20 Principal Components and are input into a Multilayer Perceptron model. The dust particle model's accuracy ranges from 69% to 82%, while the CO model achieves accuracy between 76% and 95%.

2. PolluSenseCheck system. A TinyML-based device has been prototyped to monitor ambient air quality and assess the health of its sensors. This device is equipped with a temperature and humidity sensor, an OLED screen, and an ultra-low-powered network-capable microcontroller with 512 kB SRAM. The deployed model accuracy ranges from 66% to 88% for dust particles and 74% to 94% for CO. The model sizes are 253.360 kB for CO models and 254.252 kB for dust models. The models consume a maximum of 176 Bytes of heap space and 6288 Bytes of stack space.

3. Intercept sensor fault detection algorithm. We present an algorithm to detect whether a sensor might be faulty by comparing the real and predicted AQI classes for a pollutant. The algorithm has been validated using extensive simulation experiments, and the error-catching rate can be adjusted by modifying four hyperparameters set by the user. Ideal cases of these hyperparameters have also been discussed.

This paper is organized as follows: Section II discusses existing works in the field of cost-effective air quality monitoring devices and their challenges, as well as current air quality and environmental monitoring techniques, including their applications with TinyML. Section IV describes the data acquisition process and provides insights into the data. Section V outlines the data preprocessing methodology. Section VI presents the TinyML-based PolluSenseCheck system, along with its models, algorithms, and hardware implementation. Section VII offers insights and discussions on the results obtained from the model, as well as insights on the algorithm

via simulations. Finally, Section VIII discusses the discourse and future work for this research.

II. RELATED WORK

This section discusses existing work on cost-effective air quality monitoring systems, the evaluation of low-cost sensors, and air quality prediction methods both dependent and independent of TinyML.

A. Low-Cost Air Quality Monitoring Systems

Numerous low-cost air quality monitoring systems have been developed, ranging from mobile [10] to static deployments [11]–[13]. These systems generally supplement existing high-cost air quality monitoring frameworks to achieve greater granularity [12] or monitor indoor air quality [10], [13]. Consequently, in addition to standalone sensing systems, participatory sensing systems [10], [12], systems adhering to the IEEE 1451 [11] standard for standardized development, and indoor-outdoor adaptive systems [10] have also been developed.

B. Evaluation of Low Cost Sensors & Calibration

Studies conducted both in the lab [14] and in the field [15], [16] demonstrate that low-cost sensors are less accurate than reference-grade ones, especially as pollutant particle size increases [15]. Longer evaluation studies have reported sensor accuracy drift in low-cost sensors after 15 to 18 months of operation due to dust accumulation and sensor wear [16]. Furthermore, discussions have addressed concept drift in recalibration models [17] and the necessity to retrain them for long-term accuracy. Due to the inherent inaccuracy of uncalibrated low-cost sensors, periodic calibration with reference-grade sensors is essential in such sensing systems. This is typically achieved through lab-based calibration [12], [18], co-location [19], field calibration [20], [21], or regression [20]–[22], using raw signals or meteorological factors such as temperature and humidity [17], [22].

C. Machine Learning based Air Quality Prediction

Numerous studies focus on air quality prediction [23]–[25], primarily forecasting pollutant concentrations for 24-48 hours, with some predicting the Air Quality Index as a classification task [26], [27]. Machine learning models utilize past pollutant concentrations and meteorological factors like temperature and humidity [23], [24], [26], [27], along with urban features such as traffic density [25]. Datasets are mostly sourced from public air quality [23], [25] and traffic monitoring systems [25]. Common methods include linear regression, decision trees, and random forests [23], [26], [27], as well as deep learning models like MLPs and CNNs [23]–[26]. While neural networks perform well, SVMs and random forests can match or exceed their accuracy.

D. TinyML Based Techniques for Environment Monitoring

Various TinyML techniques have been employed to predict aspects of ambient air and its pollutants, including gas leakage detection [28], vehicular emission monitoring [29], multimodal classifications of pollution levels [30], and enhancing the accuracy of low-cost sensors using gas resistance [31]. Additionally, models have utilized raw sensor readings along with ambient information like temperature and humidity [32], predicted Air Quality Index classes using secondary sensing [27], and estimated power consumption while imputing missing data [24]. TinyML platforms such as EdgeImpulse [28], MicroMLGen, M2CGen [27], [29], EloquentTinyML [24], and TensorFlow Lite [30] have been used to deploy them on resource-constrained boards, enabling intelligence at the edge.

Key takeaways. Several low-cost environmental monitoring systems, particularly for air quality, have been developed. However, low-cost sensors experience drift over time and require regular calibration. To determine if a sensor is becoming faulty without external validation, secondary sensing can predict air quality using factors beyond the pollutant sensor itself. Various machine learning techniques on resource-constrained boards can be leveraged to deploy this secondary sensing model onto the proposed device.

III. OBJECTIVES

Concerning the above discussions and existing literature, the following objectives are proposed:

- (Q1.) Can a machine learning model be prepared that uses local temperature, humidity, weather factors, temporal factors, and geospatial factors to predict the AQI class for pollutants such as dust particles and CO through secondary sensing?
- (Q2.) Can the secondary sensing model be developed for each city individually across various cities in India, each with a unique pollution profile?
- (Q3.) Can the secondary sensing mechanism be used in tandem with the real air pollutant sensors to formulate an algorithm that can quantify sensor health and indicate when it might be faulty without the need for any external validation module?
- (Q4.) Can a consumer-facing device be created using an ultra-low-powered microcontroller board and low-cost sensors that measure outdoor air quality and is also capable of applying TinyML for the self-health check algorithm, so that it can alert users when the sensors are faulty?

IV. DATASET

This section discusses about the data collection methodology, as well as provides some insights on the collected data.

A. Data Collection

As per a 2019 report on national air quality standards [33], cities were categorized by four attributes: (a) Coastal, (b)

Industrial, (c) Populations over one million, and (d) Non-Attainment (exceed National Ambient Air Quality Standards). They are also divided into North, South, East, North-East, West, and Central zones. Based on this classification, 14 cities were initially selected for the dataset. The city tier was also considered: Tier 1 cities have a population of 100,000 and above, Tier 2 cities have a population between 50,000 and 99,999, and Tier 3+ cities have a population over 20,000. Details of the cities are provided in Table I. Data was col-

City/Town	Region	Coastal city	Industrial city	Million plus city	Non attainment city	Tier of city	Used in final dataset?
Tirupati	South	✓	×	×	×	3+	✓
Vishakhapatnam	South	✓	✓	✓	✓	2	✓
Guwahati	North-East	×	×	×	✓	2	×
Delhi	North	×	✓	✓	✓	1	✓
Mangalore	South	✓	✓	×	×	3+	×
Mumbai	West	✓	×	✓	✓	1	✓
Nashik	West	×	✓	✓	✓	2	×
Byrnihat	North-East	×	✓	×	✓	3+	×
Shillong	North-East	×	×	×	×	3+	✓
Chennai	South	✓	×	✓	×	1	×
Agra	North	×	✓	✓	✓	2	×
Dehradun	North	×	×	×	✓	2	✓
Durgapur	East	×	✓	×	✓	2	✓
Kolkata	East	×	×	✓	✓	1	✓

TABLE I: Selected cities for analysis, based on diversity across types, tiers, and regions. Due to missing data for some features, certain cities had to be excluded.

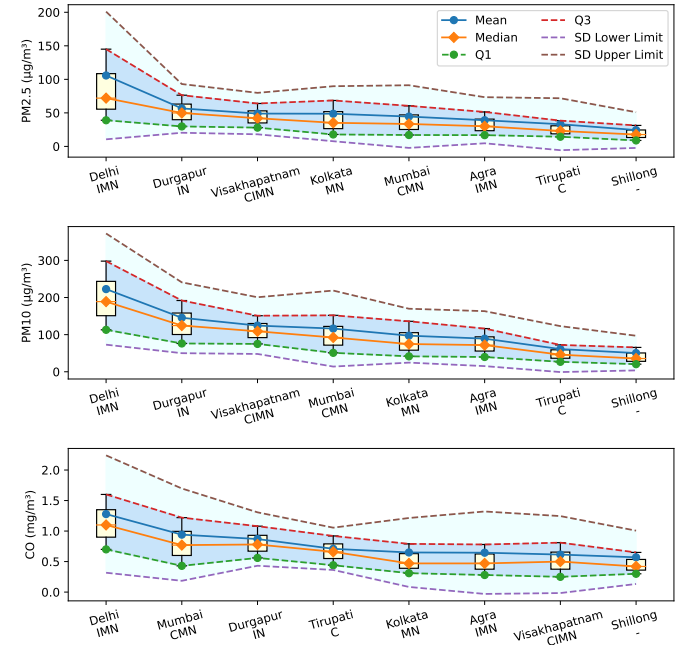


Fig. 2: The mean, median, standard deviation range, and interquartile range for the three selected pollutants across the eight cities, with cities ranked in descending order by mean values for each pollutant. Here, C = Coastal City, I = Industrial City, M = Million-plus Population, N = Non-attainment City.

lected from the Live Air Quality Data of Monitoring Stations Portal provided by the Central Control Room for Air Quality Management, Central Pollution Control Board, India. Datasets for the 14 cities were obtained for both 2022 and 2023, at 15-minute intervals.

B. Descriptive Analysis

Figure 2 shows Delhi as the most polluted city with the highest values across all pollutants and metrics. Durgapur, an industrial city, ranks second in mean and median for PM 2.5 and PM 10, and has a high interquartile range for PM 10. Mumbai, a traffic-heavy metropolis, ranks second in mean, standard deviation, and interquartile range for CO, PM 10, and PM 2.5. Kolkata ranks second in interquartile range. Shillong, a high-rainfall city, has the lowest values across all pollutants, except for CO standard deviation, where Tirupati ranks lowest.

Upon ranking each month, season, time cluster, weekday, and hour for each city using Borda count, notable patterns and deviations emerged. The deviations for each city, calculated with the Spearman Foot Rule Distance, are shown in Figure 3. Pollutant concentrations exhibit seasonal trends, with season 0 having the highest values, followed by seasons 1 and 2. Tirupati often deviates from these patterns. CO trends are violated more frequently than dust particle trends, particularly in Mumbai, Kolkata, Visakhapatnam, and Tirupati, though these deviations are minor. Winter months (November to January) show the highest pollutant levels, while monsoon months (July to September) have lower levels; Tirupati and Mumbai display notable deviations in PM 10 and CO. Thursdays and Fridays experience the highest pollutant concentrations, while Sundays and Mondays are the cleanest, especially Sundays. Deviations from the trend in standard deviation and interquartile range are observed in Shillong, Visakhapatnam, Delhi, Tirupati, and Durgapur. Time clusters 1 and 0 show higher PM 2.5 levels, while clusters 2 and 3 show lower values. PM 10 concentrations peak in clusters 1 and 3, with similar trends for CO, though Mumbai and Visakhapatnam deviate. Night-time (21:00–23:00) and early morning (07:00–08:00) hours have higher pollutant levels, while afternoons (14:00–16:00) are cleaner. Most cities follow these trends, except Tirupati, Kolkata, Shillong, Mumbai, and Durgapur, which deviate in median PM 2.5 concentrations.

Pairwise correlation analysis of PM 2.5 with features $f_1 - f_6$ —shows that temperature has a high negative correlation, followed by wind speed, humidity, and rainfall. Wind direction and barometric pressure correlate positively with PM 2.5. Tirupati deviates from this trend, with barometric pressure showing the most negative correlation and temperature a slight positive correlation. Coastal cities exhibit strong negative correlations, while industrial cities show a strong negative correlation with temperature and a positive correlation with barometric pressure. This trend is similar to cities with populations over a million and non-attainment cities. The trend for PM 10 mirrors that of PM 2.5. CO also shows a strong negative correlation with temperature, followed by wind speed, humidity, and rainfall, with positive correlations for wind

direction and barometric pressure. Tirupati again deviates, while coastal cities and industrial cities follow similar trends.

V. DATA PREPROCESSING

This section discusses the feature engineering and data preprocessing performed on the dataset. The input to each model comprises 53 features, f_1 to f_{53} , as listed in Table II. The output of each model is an AQI class on a scale from 1 to 6, with the classes denoting good, satisfactory, moderate, poor, very poor, and severe air quality respectively. Our observations indicate that CO has not exceeded AQI 3, and dust particles have rarely reached AQI 6. Hence, for the dust model, AQI 5 and 6 labels were combined into AQI 5. Additionally, the CO model becomes a three-class classification problem, predicting AQI classes 1, 2, or 3.

Feature ID	Feature description	Data source
$f_1 - f_6$	Atmospheric Temperature, Relative Humidity, Wind Speed, Direction of Wind, Total Rainfall, and Barometric Pressure	CPCB [34]
$f_7 - f_{11}$	The month, weekday, hour, season, and timecluster of the recorded datapoint	Engineered from timestamp
$f_{12} - f_{13}$	Latitude and longitude of the station's location	Various mapping services
$f_{14} - f_{17}$	Boolean columns indicating if the station is in a coastal city, industrial city, city over one million population, or non-attainment city	National Ambient Air Quality Monitoring Standards report, 2019 [33]
$f_{18} - f_{20}$	Boolean columns for Tier 1, Tier 2, Tier 3, or Tier 4 cities	Wikipedia
$f_{21} - f_{53}$	Land coverage by human-made structures, natural land, highways, roads, water bodies, parks, schools, medical facilities, tourist attractions, or airports within 1.5 km, 1 km, or 0.5 km of the station	Collected from Google Maps Static API

TABLE II: Description and source of the feature list

A. Feature Engineering

Based on previous literature [26], [27], the sought features from CPCB data were: (a) $PM_{2.5}$ ($\mu g/m^3$), (b) PM_{10} ($\mu g/m^3$), (c) CO (mg/m^3), (d) AT ($^{\circ}C$), (e) RH (%), (f) WS (m/s), (g) WD (deg), (h) $TOT-RF$ (mm), (i) BP ($mmHg$). A preliminary analysis revealed that several stations lacked one or more specified features, reducing the dataset to 8 cities (as given in Table I).

Temporal features are listed as $f_7 - f_{11}$ in Table II. The *season* feature has three labels: Season 0 (October to December), Season 1 (March to July), and Season 2 (August to September). The *timecluster* feature includes four labels: Timecluster 0 (2200–0600), Timecluster 1 (0700–0900), Timecluster 2 (1000–1600), and Timecluster 3 (1700–2100).

Considering the portable nature of personal air quality monitors and existing literature as discussed in section II-C, landmark features were extracted using the Google Static Maps API. Points of interest were color-coded and extracted from maps around each station using radii of 1.5 km, 1 km, and 0.5 km. The percentage of landmark features corresponds to the proportion of each landmark's color, as listed in $f_{21} - f_{53}$ in Table II. It is interesting to note that for both dust and

CO, the air quality categories are imbalanced, necessitating the oversampling of data points using the Synthetic Minority Oversampling Technique (SMOTE). Additionally, the dataset ranges from small fractional values close to zero to thousands, so it was scaled between 0 and 1. Lastly, we reduce the number of features, by employing Principal Component Analysis (PCA), resulting in the transformed dataset X_T .

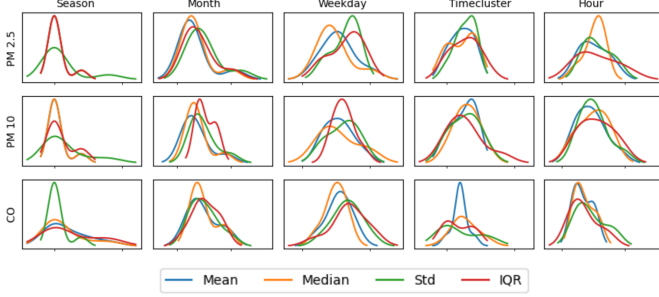


Fig. 3: Distribution of deviations from the general trend, increasing from left to right. The lack of deviation plots for the mean and median of pollutants by season reflects total alignment with the general trend

VI. TINYML-BASED POLLUSENSECHECK SYSTEM

This section discusses the TinyML model development and sensor fault check algorithm. The notations used for this section have been tabulated in Table III.

Notation	Description
P	Set of pollutants, $\{\text{Dust}, \text{CO}\}$
E	Set of environmental factors, $\{\text{Temperature}, \text{Humidity}, \text{PM}_{2.5}, \text{PM}_{10}, \text{CO}\}$
DS	Dataset consisting of sensor data
X	The input vector, contains 53 input features: $f_1 - f_{53}$. The details of the features have been defined in Table II.
X_T	The input vector X , scaled to the range $[0,1]$, and transformed by PCA.
Y_p	Real AQI class for the pollutant $p \in P$, directly measured from sensors
Tiny-Model-Env _p	The pre-trained and quantized model for predicting the $\hat{Y}_{p,x}$, deployed onto the microcontroller board
$\hat{Y}_{p,x}$	The predicted AQI class for the pollutant $p \in P$, with input vector X .
f_p	The fault score associated with the sensor of the pollutant $p \in P$. The fault score is a measure of the amount of faulty records encountered by the sensor.
α_p	Hyperparameter for <i>Intercept</i> algorithm. It is the growth rate of f_p , for pollutant $p \in P$.
β_p	Hyperparameter for <i>Intercept</i> algorithm. It is the decay rate of f_p for pollutant $p \in P$.
τ_p	The sensor fault tolerance for a single record, applied to the sensor of pollutant $p \in P$. It defines the permissible deviation of $\hat{Y}_{p,x}$ from Y_p in AQI classes before marking the record as faulty and updating the fault score f_p .
θ_p	Hyperparameter for <i>Intercept</i> algorithm. It is the sensor fault threshold for the sensor of pollutant $p \in P$ and signifies how much fault score f_p value the sensor should reach before it is tagged as "might be faulty"
$\mathcal{F}_p$	The binary fault status for the sensor measuring the pollutant $p \in P$

TABLE III: Notation table

A. TinyML Model Design

As discussed in Sec. II-C, the existing literature [26] shows that air quality varies over very small distances. Due to this highly granular nature of air pollution, models need to be location-specific. However, for portable air quality monitors, training on a single locality is insufficient, as the models may fail to capture trends in other areas. As illustrated in Figure 3, there is significant variation in generalized trends across cities. To balance portability and accuracy, a city-specific multilayer perceptron model is proposed, with two models per city — one for dust particles and one for CO.

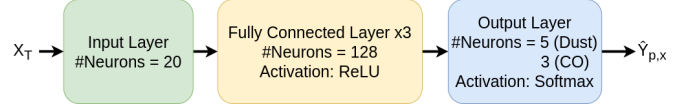


Fig. 4: TinyML model architecture

The models aim to predict $\hat{Y}_{p,x}$ given X_T . Figure 4 illustrates the proposed architecture, a three-layer multilayer perceptron. The input layer has twenty neurons corresponding to the twenty principal components, followed by three fully connected hidden layers with 128 neurons each, and an output layer with c neurons, where c is the number of air quality classes for the monitored pollutant. For instance, CO has $c = 3$ classes, while dust particles have $c = 5$ classes.

For each city, the city-specific dataset was split into training and test sets, with 20% reserved for testing in a stratified manner. From the training set, 20% was reserved for validation. The model was built using the TensorFlow library with the Keras API, which provides deep learning operations and libraries for converting models into their TinyML counterparts.

To enable operation on resource-constrained devices, quantization-aware training was performed to reduce quantization loss. The model was converted to *TensorFlow Lite* using default quantization, where parameters are rendered as integers — yielding *TinyModelEnv_p*. The operations use a mix of integers and *float32*, reverting to *float32* when needed. After quantization, the model was evaluated with the *TensorFlow Lite* interpreter and saved in .tflite format. Finally, the saved model was converted into an array of weights for seamless integration into device firmware.

B. Intercept Sensor Fault Detection Algorithm

The system aims to compare the real air quality index class of a pollutant, using on-device sensors, and also perform secondary sensing to predict the same, independent of the sensor. A significant amount of deviation between the two values, for a significant amount of time, will cause the device to report that the sensor might be faulty. The overall main function has been described in Algorithm 1. It has been subdivided into further smaller steps.

The system first senses environmental factors $e \in E$ using on-device sensors and calculates Y_p for all $p \in P$. An input vector X , comprising features $f_1 - f_{53}$, is created. Temperature and humidity are obtained from on-board sensors, while

Algorithm 1 PolluSenseCheckMain

Main function of the firmware. This function continuously senses and updates the fault score and fault status of sensors. This function also stores all data, and updates the display of the device.

```

1: Initialize  $\mathcal{F}_p \leftarrow 0, f_p \leftarrow 0 \forall p \in P$ 
2: Declare  $\alpha_p, \beta_p, \tau_p, \theta_p, X_{\max}, X_{\min}$ 
3: while True do
4:   realSense(DSe  $\forall e \in E$ )
5:    $Y_p \leftarrow \text{calculateAQI}(p), \forall p \in P$ 
6:    $X \leftarrow \text{acquireData}(f_1 - f_{53})$ 
7:   for  $\forall p \in P$  do
8:      $\hat{Y}_{p,x} \leftarrow \text{predictAQI}(X, \text{TinyModelEnv}_p)$ 
9:      $f_p, \mathcal{F}_p \leftarrow \text{Intercept}(\alpha_p, \beta_p, \tau_p, \theta_p, f_p, Y_p, \hat{Y}_{p,x})$ 
10:    reportFault( $\mathcal{F}_p$ )
11:   end for
12: end while

```

other features are engineered from timestamps or sourced via network calls. X is scaled to $[0, 1]$ and transformed into 20 principal components using pre-calculated eigenvectors, yielding X_T . TinyModelEnv_p for all $p \in P$ are loaded onto the microcontroller. TinyModelEnv_p then predicts the pollutant's Air Quality Index class, $\hat{Y}_{p,x}$, from X_T . At the core of the system is the *Intercept* sensor fault detection algorithm, which detects faulty sensors without external tools, as described in Algorithm 2. A record is marked faulty if the difference between Y_p and $\hat{Y}_{p,x}$ exceeds a threshold, determined by the hyperparameter τ_p . This threshold is tuned based on the number of classes predicted and the model's error margin. For $\text{TinyModelEnv}_{\text{dust}}$, which often mispredicts by one AQI class, τ_p is set to 2, allowing a relaxation of 1 class. In contrast, for CO, a lower τ_p is used, reflecting in lieu of its higher accuracy and fewer classes. Each sensor is assigned a fault score f_p , which increases when a faulty record is detected and decreases otherwise, as shown in Equation (1).

$$f_p = \begin{cases} (\alpha_p \times f_p) + 1, & \text{if } |\hat{Y}_{p,x} - Y_p| < \tau_p \\ (\beta_p \times f_p), & \text{otherwise} \end{cases} \quad (1)$$

The hyperparameters α_p and β_p control the rate of increase and decrease of f_p . Once f_p exceeds the threshold θ_p , the sensor is marked as faulty. α_p controls how quickly f_p increases for faulty records. A low α_p causes slower detection, while a higher value makes the algorithm more sensitive, including to outliers, as it accelerates the rise in f_p , bringing it closer to the threshold. β_p controls how much of f_p is retained when a record is not faulty. A β_p of zero resets f_p after each non-faulty record, delaying fault detection. A β_p of 1 retains f_p , making the system more sensitive to sparse outliers. β_p should be tuned in such a way that it should neither take too long to reach θ_p nor cause false alarms. The value of θ_p also affects fault detection sensitivity. A lower θ_p can cause false alarms, while a higher θ_p delays fault recognition. The impact of tuning the four hyperparameters— $\alpha_p, \beta_p, \tau_p$, and

Algorithm 2 Intercept

For a pollutant $p \in P$, updates the fault score by comparing the real and predicted AQI class. If the fault score of a pollutant is above a threshold, it is labelled faulty.

Input: $\alpha_p, \beta_p, \tau_p, \theta_p, f_{\text{init}_p}, Y_p, \hat{Y}_{p,x}, p \in P$
Output: $f_{\text{fin}_p}, \mathcal{F}_p, p \in P$

```

1: if  $|\hat{Y}_{p,x} - Y_p| \geq \tau_p$  then
2:    $f_{\text{fin}_p} \leftarrow (\alpha_p \times f_{\text{init}_p}) + 1$ 
3: else
4:    $f_{\text{fin}_p} \leftarrow (\beta_p \times f_{\text{init}_p})$ 
5: end if
6: if  $f_{\text{fin}_p} \geq \theta_p$  then
7:    $\mathcal{F}_p \leftarrow 1$  //Sensor might be faulty
8: else
9:    $\mathcal{F}_p \leftarrow 0$  //Sensor is not faulty
10: end if
11: return  $f_{\text{fin}_p}, \mathcal{F}_p$ 

```

θ_p —on the algorithm's sensitivity is discussed in Section VII. If f_p exceeds θ_p , the sensor is likely faulty, and the fault status $\mathcal{F}_p$ is set to 1; otherwise, it remains 0. The *Intercept* algorithm returns an ordered pair of the updated f_p and $\mathcal{F}_p$. Finally, the system checks $\mathcal{F}_p$ for each sensor. If any sensor has $\mathcal{F}_p = 1$, the user is prompted to replace or calibrate the sensor.

C. Hardware Implementation

The proposed device shown in Figure 5 has been developed that monitors air quality, provides insights via a web dashboard, and checks sensor health, alerting users as needed. It includes sensors for dust (PM 2.5/PM 10) and CO, an SD card for storage, and an OLED display, all connected to a Wi-Fi-enabled microcontroller. TensorFlow Lite Micro models predict AQI classes, while an HTTP request retrieves meteorological and geographical data for AQI calculations. The *Intercept* algorithm updates fault scores. Data is stored, and current air quality and sensor status are shown on the OLED screen. The system uses the ESP32-S3-WROOM1 microcon-

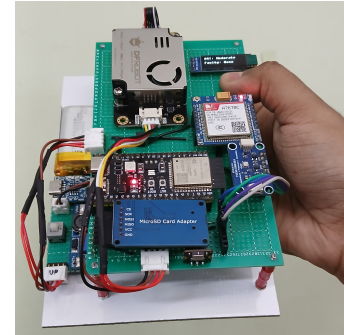


Fig. 5: Prototype of PolluSenseCheck system

troller with 512KB SRAM, 8MB PSRAM, Wi-Fi, Bluetooth, and 27 GPIOs for real-time inference. Sensors include the

BME688 (temperature/humidity), DFRobot SEN0460 (dust), and a DFRobot Fermion MEMS CO sensor. A rechargeable battery enhances portability. A sim module provides network connectivity in places of static deployment of the device, where Wi-Fi is unavailable. Lastly, firmware is written in C/C++ using the ESP-IDF framework, with a dashboard displaying pollutant levels and sensor status alerts.

VII. RESULT ANALYSIS

This section discusses the performance of the models developed for dust as well as CO. It demonstrates the performance of *Intercept* algorithm and the effect of parameter tuning on the sensitivity of sensor fault detection.

Feature analysis. The features and pollutants differ across cities, as discussed in Section IV-B. Geographical features are less significant when models are trained on city-specific data. However, PCA on aggregate data from all 8 cities reveals geographical features to be an important determinant of pollution. The top three features from the first ten principal components, explaining 75% of the variance, included key meteorological factors like temperature and humidity. Significant temporal features such as season, month, and hour are also observed. Among geographical features, station longitude was significant, with coastal cities exhibiting lower pollution levels due to their moderate climate, while industrial cities showed poorer air quality. It is also observed that the tier of city also affects air quality. Landmark features like parks, highways, and road types were important; parks indicate cleaner areas and traffic patterns influence air quality.

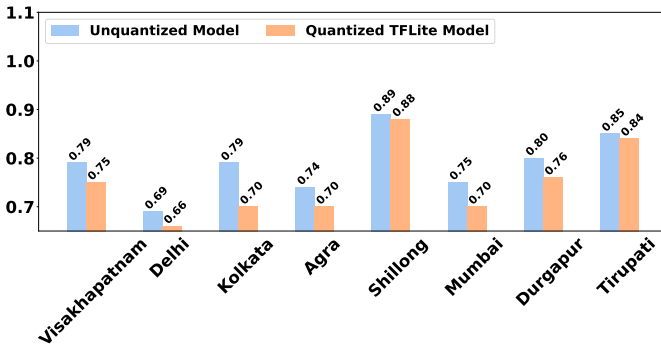


Fig. 6: Comparison of accuracies for the unquantized/base models for dust particle AQI class prediction, with their quantized counterparts converted to TensorFlow Lite

Model performance analysis. Two models were prepared for each of the eight cities — one for dust and another for CO. The accuracies of unquantized and quantized models for dust are shown in Figure 6, while CO model accuracies are displayed in Figure 7. For dust, lower accuracies were observed in Delhi, Agra, and Mumbai, with Delhi having the lowest at 69%. These cities have a high spatial and temporal variance in dust concentration, on account of the cities being larger and data being collected from multiple stations. Higher

accuracies in Visakhapatnam, Durgapur, Shillong, and Tirupati are attributed to the availability of data from a single station and lower variance, as seen in Figure 2. Shillong, with the lowest variance, achieved the highest accuracy at 0.89. However, Kolkata, another metropolis, had lower accuracy in the quantized model, similar to Delhi, Agra, and Mumbai.

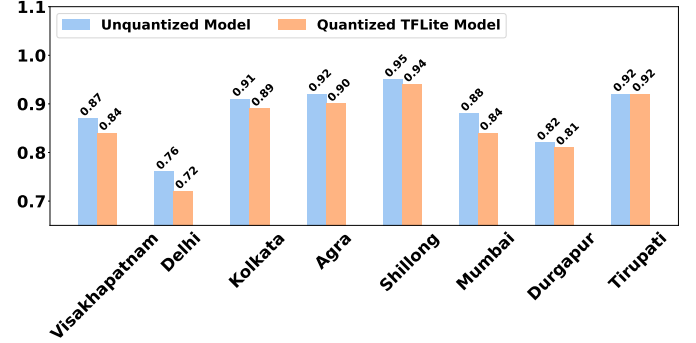


Fig. 7: Comparison of accuracies for the unquantized/base models for CO AQI class prediction, with their quantized counterparts converted to TensorFlow Lite

As mentioned in Section V, dust AQI prediction is treated as a five-class classification, while CO is a three-class problem. Fewer classes lead to higher CO prediction accuracy compared to dust models.

Quantization slightly reduces model accuracy, with Delhi dropping to 0.66 and Shillong to 0.88. Kolkata's accuracy fell the most, from 0.79 to 0.70, likely due to its metropolitan complexity. Precision, recall, and F1-score closely follow accuracy values. Models performed worst for dust AQI classes 4 and 5 but best for classes 2 and 3. CO models excelled at predicting class 1, reflecting the original data distribution. It was also observed that Agra, Visakhapatnam, Tirupati; and Kolkata, Mumbai, and Durgapur formed two distinct clusters based on PM 2.5's variance.

Previous work	Model used	City evaluated	Model test accuracy	Deployed in microcontroller?	Deployed model test accuracy
Sharma et al. [26]	Random Forest	Durgapur	0.76	×	-
Datta et al. [27]	XGBoost	Durgapur	0.76	✓	0.72
PolluSense-Check	Multilayer Perceptron	Durgapur	0.80	✓	0.76

TABLE IV: Comparison of existing work on secondary sensing of AQI classes for dust particles

Comparative study. Existing work on predicting AQI class for pollutants using secondary sensing [26] [27] has modeled data on dust particles. Comparing the results with the current model, as tabulated in Table IV it has been observed that the model provides a better test accuracy, and when deployed to the microcontroller, it provides accuracy comparable to

existing literature. These works have not addressed the concentration of CO, though it is a major air pollutants.

Model performance. Since the model architecture remains constant for all cities, the size of the deployed Tensorflow Lite Micro models are the same. The size of the CO predictors is 253.360 kB for each city, and the size of the dust particle predictors is 254.252 kB for each city. To initialize the model, a heap space of 176 Bytes, and a stack space of 6016 Bytes was consumed. PCA transformation consumes 6288 Bytes of stack, but no heap memory due to usage of static variables. Performing inference for each model consumes 64 Bytes of heap memory and 704 Bytes of stack memory.

Behavior analysis of *Intercept* fault detection algorithm.

Due to the lack of available data on dust particle sensor drift, simulated erroneous data was generated. This was done by introducing Gaussian noise into Durgapur's dust particle readings from the dataset, with a linearly increasing variance to represent a high rate of error growth. The erroneous sensor data is treated as Y_{dust} , while $\hat{Y}_{\text{dust}}$ is predicted from the corresponding meteorological, spatial, and temporal data. For the simulation, the hyperparameters are initialized as $\alpha_{\text{dust}} = 1$, to increment the entire fault score for an earlier fault detection; $\beta_{\text{dust}} = 0.95$, so that only 5% of the fault score is lost, ensuring a retention rate of 95%; $\tau_{\text{dust}} = 2$, to allow the algorithm a maximum difference of one AQI category for dust, accounting for the error of the Durgapur model; and $\theta_{\text{dust}} = 10$, to enable early detection of sensor faults. The f_{dust} is calculated for each record by updating it according to Algorithm 2. The values of $|Y_{\text{dust}} - \hat{Y}_{\text{dust}}|$ for each record and the corresponding f_{dust} are shown in Figure 8. The inset plot zooms in on the range where f_p reaches the threshold. When the difference $|Y_{\text{dust}} - \hat{Y}_{\text{dust}}|$ lies between 0 and 1, the value of f_{dust} does not increase. When the value of $|Y_{\text{dust}} - \hat{Y}_{\text{dust}}|$ crosses τ_{dust} , the value of f_{dust} increments by 1. Otherwise, the value of f_{dust} decreases by 5% every iteration and approaches 0. If f_{dust} crosses the value of θ_{dust} , the system indicates the presence of an error. Notably, due to the increasing nature of the simulated error, the number of times f_{dust} crosses θ_{dust} increases with each record.

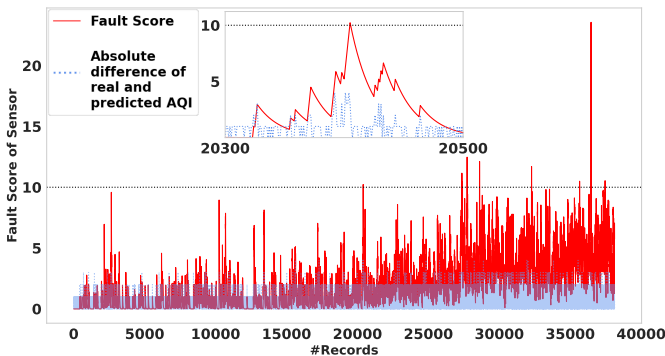


Fig. 8: Simulation results of the *Intercept* algorithm

To demonstrate the nature of hyperparameters and the effect of tuning each hyperparameter, the hyperparameters are initialized to $\alpha_{\text{dust}} = 1$, $\beta_{\text{dust}} = 0.5$, $\tau_{\text{dust}} = 2$, and $\theta_{\text{dust}} = 10$.

By keeping these hyperparameters constant and varying one hyperparameter at a time, we obtain results shown in Figure 9. It was observed that f_p is unable to reach the threshold value unless α_{dust} is very close to 1; However, once it exceeds 1, f_p reaches the threshold more quickly. Similarly, a lower value of β_{dust} ensures that the fault is not detected too early, reducing the chance of false alarms. However, this decreased sensitivity may also cause the system to remain prone to faulty sensors for a longer duration. For τ_{dust} , a lower value allows for

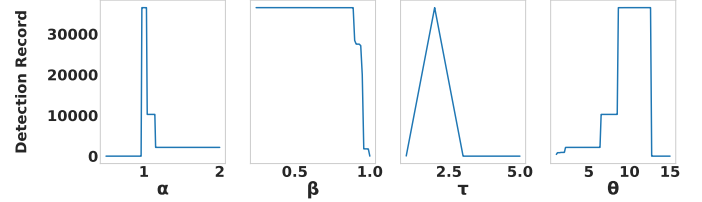


Fig. 9: Effect of hyperparameters on how quickly the *Intercept* algorithm detects sensor fault

earlier sensor fault detection. However, after a certain value (in this case, 2), the error becomes undetectable due to excessive relaxation. This behavior is also visible with θ_{dust} , whose value is observed to plateau at the maxima for the range of 9 to 13. The sensor fault can be detected at a maximum of 37,000 records due to the large spike in the fault score, as observed in Figure 8.

VIII. FUTURE WORK AND CONCLUSION

This work discusses a system for sensor self-health checks using a secondary sensing model without specialized equipment or periodic calibration. Separate multilayer perceptron models were trained for each of the eight cities, with two models per city—one for dust and one for CO. Citywide models, suitable for portable devices or areas without nearby monitoring stations, face slight accuracy reductions, while station-specific models offer higher accuracy. Static devices, once sufficient data is collected, improve secondary sensing. These models were quantized and converted into TinyML models via TensorFlow Lite. The models were implemented on an ultra-low-power microcontroller with cost-effective sensors as well. The *Intercept* fault detection algorithm was introduced and showed effective sensor fault detection in simulations. Future work will focus on enhancing model accuracy by expanding the dataset spatially and feature-wise, with the goal of a unified model for nationwide or cluster-based use. Incorporating traffic concentration data, though limited in countries like India, could improve predictions. Additionally, real sensor drift data will be collected to optimize the *Intercept* algorithm.

IX. ACKNOWLEDGEMENTS

This publication is an outcome of the R&D work undertaken in the Project "i-PolluSense: Intelligent Air & Noise Pollution Sensing and Alert Companion" funded by AI4ICPS I Hub Foundation, IIT Kharagpur, West Bengal, India for funding our research work. We also acknowledge the public air pollution data provided by the Central Pollution Control Board, India.

REFERENCES

- [1] F. Kelly and J. Fussell, "Air pollution and public health: emerging hazards and improved understanding of risk," *Environmental geochemistry and health*, vol. 37, pp. 631–649, 2015.
- [2] A. van Erp, F. Kelly, K. Demerjian, C. Pope, and A. Cohen, "Progress in research to assess the effectiveness of air quality interventions towards improving public health," *Air Quality, Atmosphere & Health*, vol. 5, pp. 217–230, 2012.
- [3] S. Henschel, R. Atkinson, A. Zeka, A. Le Tertre, A. Analitis, K. Katsouyanni, O. Chanel, M. Pascal, B. Forsberg, S. Medina *et al.*, "Air pollution interventions and their impact on public health," *International journal of public health*, vol. 57, pp. 757–768, 2012.
- [4] R. Dubey, S. Bharadwaj, M. Zafar, and S. Biswas, "Collaborative air quality mapping of different metropolitan cities of india," *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, vol. 43, pp. 87–94, 2021.
- [5] M. Brauer, S. Guttikunda, K. Nishad, S. Dey, S. Tripathi, C. Weagle, and R. Martin, "Examination of monitoring approaches for ambient air pollution: A case study for india," *Atmospheric Environment*, vol. 216, p. 116940, 2019.
- [6] P. Rahi, S. Sood, R. Bajaj, and Y. Kumar, "Air quality monitoring for smart ehealth system using firefly optimization and support vector machine," *International Journal of Information Technology*, vol. 13, no. 5, pp. 1847–1859, 2021.
- [7] L. Morawska, P. Thai, X. Liu, A. Asumadu-Sakyi, G. Ayoko, A. Bartonova, A. Bedini, F. Chai, B. Christensen, M. Dunbabin *et al.*, "Applications of low-cost sensing technologies for air quality monitoring and exposure assessment: How far have they gone?" *Environment international*, vol. 116, pp. 286–299, 2018.
- [8] F. Karagulian, M. Barbiere, A. Kotsev, L. Spinelle, M. Gerboles, F. Lagler, N. Redon, S. Crunaire, and A. Borowiak, "Review of the performance of low-cost sensors for air quality monitoring," *Atmosphere*, vol. 10, no. 9, p. 506, 2019.
- [9] X. Liu, R. Jayaratne, P. Thai, T. Kuhn, I. Zing, B. Christensen, R. Lamont, M. Dunbabin, S. Zhu, J. Gao *et al.*, "Low-cost sensors as an alternative for long-term air quality monitoring," *Environmental research*, vol. 185, p. 109438, 2020.
- [10] P. Kortoçi, N. H. Motlagh, M. A. Zaidan, P. L. Fung, S. Varjonen, A. Rebeiro-Hargrave, J. V. Niemi, P. Nurmi, T. Hussein, T. Petäjä, M. Kulmala, and S. Tarkoma, "Air pollution exposure monitoring using portable low-cost air quality sensors," *Smart Health*, vol. 23, Mar. 2022. [Online]. Available: <http://dx.doi.org/10.1016/j.smhl.2021.100241>
- [11] N. Kularatna and B. H. Sudantha, "An environmental air pollution monitoring system based on the ieec 1451 standard for low cost requirements," *IEEE Sensors Journal*, vol. 8, no. 4, pp. 415–422, Apr. 2008. [Online]. Available: <http://dx.doi.org/10.1109/JSEN.2008.917477>
- [12] S. Brienza, A. Galli, G. Anastasi, and P. Bruschi, "A low-cost sensing system for cooperative air quality monitoring in urban areas," *Sensors*, vol. 15, no. 6, pp. 12 242–12 259, May 2015. [Online]. Available: <http://dx.doi.org/10.3390/s150612242>
- [13] P. Karmakar, S. Pradhan, and S. Chakraborty, "Exploring indoor air quality dynamics in developing nations: A perspective from india," *ACM Journal on Computing and Sustainable Societies*, 2024.
- [14] A. Baldelli, "Evaluation of a low-cost multi-channel monitor for indoor air quality through a novel, low-cost, and reproducible platform," *Measurement: Sensors*, vol. 17, p. 100059, Oct. 2021. [Online]. Available: <http://dx.doi.org/10.1016/j.measen.2021.100059>
- [15] I. Stavroulas, G. Grivas, P. Michalopoulos, E. Liakakou, A. Bougiatioti, P. Kalkavouras, K. Fameli, N. Hatzianastassiou, N. Mihalopoulos, and E. Gerasopoulos, "Field evaluation of low-cost pm sensors (purple air pa-ii) under variable urban air quality conditions, in greece," *Atmosphere*, vol. 11, no. 9, p. 926, Aug. 2020. [Online]. Available: <http://dx.doi.org/10.3390/atmos11090926>
- [16] G.-H. Hong, T.-C. Le, J.-W. Tu, C. Wang, S.-C. Chang, J.-Y. Yu, G.-Y. Lin, S. G. Aggarwal, and C.-J. Tsai, "Long-term evaluation and calibration of three types of low-cost pm_{2.5} sensors at different air quality monitoring stations," *Journal of Aerosol Science*, vol. 157, p. 105829, Sep. 2021. [Online]. Available: <http://dx.doi.org/10.1016/j.jaerosci.2021.105829>
- [17] M. A. Zaidan, N. Hossein Motlagh, P. L. Fung, D. Lu, H. Timonen, J. Kuula, J. V. Niemi, S. Tarkoma, T. Petaja, M. Kulmala, and T. Hussein, "Intelligent calibration and virtual sensing for integrated low-cost air quality sensors," *IEEE Sensors Journal*, vol. 20, no. 22, pp. 13 638–13 652, Nov. 2020. [Online]. Available: <http://dx.doi.org/10.1109/JSEN.2020.3010316>
- [18] A. Martinez, E. Hernandez-Rodríguez, L. Hernandez, O. Schalm, R. A. González-Rivero, and D. Alejo-Sánchez, "Design of a low-cost system for the measurement of variables associated with air quality," *IEEE Embedded Systems Letters*, vol. 15, no. 2, pp. 105–108, Jun. 2023. [Online]. Available: <http://dx.doi.org/10.1109/LES.2022.3196543>
- [19] G. Coulby, A. K. Clear, O. Jones, and A. Godfrey, "Low-cost, multimodal environmental monitoring based on the internet of things," *Building and Environment*, vol. 203, p. 108014, Oct. 2021. [Online]. Available: <http://dx.doi.org/10.1016/j.buildenv.2021.108014>
- [20] L. Spinelle, M. Gerboles, M. G. Villani, M. Aleixandre, and F. Bonavitacola, "Field calibration of a cluster of low-cost available sensors for air quality monitoring. part a: Ozone and nitrogen dioxide," *Sensors and Actuators B: Chemical*, vol. 215, pp. 249–257, Aug. 2015. [Online]. Available: <http://dx.doi.org/10.1016/j.snb.2015.03.031>
- [21] D. H. Hagan, G. Issacman-Vanwertz, J. P. Franklin, L. M. M. Wallace, B. D. Kocar, C. L. Heald, and J. H. Kroll, "Calibration and assessment of electrochemical air quality sensors by co-location with reference-grade instruments," Aug. 2017. [Online]. Available: <http://dx.doi.org/10.5194/amt-2017-296>
- [22] Y. Cheng, X. Li, Z. Li, S. Jiang, Y. Li, J. Jia, and X. Jiang, "Aircloud: a cloud-based air-quality monitoring system for everyone," in *Proceedings of the 12th ACM Conference on Embedded Network Sensor Systems*, ser. SenSys '14. ACM, Nov. 2014. [Online]. Available: <http://dx.doi.org/10.1145/2668332.2668346>
- [23] Y. Zheng, X. Yi, M. Li, R. Li, Z. Shan, E. Chang, and T. Li, "Forecasting fine-grained air quality based on big data," in *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '15. ACM, Aug. 2015. [Online]. Available: <http://dx.doi.org/10.1145/2783258.2788573>
- [24] I. N. K. Wardana, S. A. Fahmy, and J. W. Gardner, "Tinyml models for a low-cost air quality monitoring device," *IEEE Sensors Letters*, vol. 7, no. 11, pp. 1–4, Nov. 2023. [Online]. Available: <http://dx.doi.org/10.1109/LSENS.2023.3315249>
- [25] D. Schurholz, S. Kubler, and A. Zaslavsky, "Artificial intelligence-enabled context-aware air quality prediction for smart cities," *Journal of Cleaner Production*, vol. 271, p. 121941, 2020.
- [26] P. Pramanik, P. Karmakar, P. K. Sharma, S. Chatterjee, A. Roy, S. Mandal, S. Nandi, S. Chakraborty, M. Saha, and S. Saha, "Aquamoho: Localized low-cost outdoor air quality sensing over a thermo-hygrometer," *ACM Trans. Sen. Netw.*, jan 2023, just Accepted. [Online]. Available: <https://doi.org/10.1145/3580279>
- [27] A. Datta, A. Pal, R. Marandi, N. Chattaraj, S. Nandi, and S. Saha, "Efficient air quality index prediction on resource-constrained devices using tinyml: Design, implementation, and evaluation," in *Proceedings of the 25th International Conference on Distributed Computing and Networking*, ser. ICDCN '24. ACM, Jan. 2024. [Online]. Available: <http://dx.doi.org/10.1145/3631461.3631956>
- [28] V. Tsoukas, A. Gkogkidis, E. Boumpa, S. Papafotikas, and A. Kakarountas, "A gas leakage detection device based on the technology of tinyml," *Technologies*, vol. 11, no. 2, p. 45, Mar. 2023. [Online]. Available: <http://dx.doi.org/10.3390/technologies11020045>
- [29] P. Andrade, I. Silva, M. Silva, T. Flores, J. Cassiano, and D. G. Costa, "A tinyml soft-sensor approach for low-cost detection and monitoring of vehicular emissions," *Sensors*, vol. 22, no. 10, p. 3838, May 2022. [Online]. Available: <http://dx.doi.org/10.3390/s22103838>
- [30] J. Botero-Valencia, C. Barrantes-Toro, D. Marquez-Viloria, and J. M. Pearce, "Low-cost air, noise, and light pollution measuring station with wireless communication and tinyml," *HardwareX*, vol. 16, Dec. 2023. [Online]. Available: <http://dx.doi.org/10.1016/j.ohx.2023.e00477>
- [31] M. F. Yousuf, T. Siddique, and M. S. Mahmud, "ibug: Ai enabled iot sensing platform for real-time environmental monitoring," in *2022 IEEE 31st Microelectronics Design & Test Symposium*. IEEE, 2022, pp. 1–7.
- [32] M. El Adoui, T. Herpoel, and B. Frénay, "Constrained tiny machine learning for predicting gas concentration with i4.0 low-cost sensors," *ACM Transactions on Embedded Computing Systems*, vol. 23, no. 3, p. 1–23, May 2024. [Online]. Available: <http://dx.doi.org/10.1145/3590956>
- [33] P. Gargava and V. Shukla, "National ambient air quality status and trends 2019," CPCB, Ministry of Environment, Forest and Climate Change, Gov. of India, Tech. Rep. NAAQMS /45/2019-2020, 2019.
- [34] "CCR — airquality.cpcb.gov.in," <https://airquality.cpcb.gov.in/ccr/#/caaqm-dash-board-all/caaqm-landing/>, [Accessed 15-09-2024].